# Rockchip SecurityBoot and AVB Operation Guide

ID: RK-KF-YF-E78

Release Version: V1.0.0

Release Date: 2026-06-08

Security Level: □Top Secret □Secret □Internal ■Public

**DISCLAIMER**

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

**Trademark Statement**

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

**All rights reserved. ©2025. Rockchip Electronics Co., Ltd.**

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

**Preface**

**Overview**

This document provides configuration methods for SecurityBoot and AVB.

**Target Audience**

This document is primarily intended for the following engineers:

Technical Support Engineers

Software Development Engineers

**Revision History**

| Version | Author | Date | Description |
| --- | --- | --- | --- |
| V1.0.0 | Wu Liangqing | 2026-06-08 | Integrated from platform-specific documents |

**Table of Contents**


**Rockchip SecurityBoot and AVB Operation Guide**

1. Supported Chips
2. Software Environment
3. SecurityBoot Operation Steps
    - 3.1 Build Environment Check
    - 3.2 Enter the u-boot Directory
    - 3.3 Code Modifications
    - 3.4 Generate Keys
    - 3.5 Build and Sign
        - 3.5.1 FIT Independent Signing
        - 3.5.2 Signing Flow
        - 3.5.3 Signing Script
    - 3.6 Build Complete Firmware
    - 3.7 Firmware Flashing
    - 3.8 Verify Fusing Success
        - 3.8.1 Check via UART Boot Log
        - 3.8.2 Check by Flashing Unsigned Loader and Uboot
4. Android Verified Boot (AVB) Operation Steps
    - 4.1 About AVB Key Management
    - 4.2 Build avbtool
    - 4.3 Generate permanent_attributes.bin
        - 4.3.1 Method 1: Android14 and Earlier (ATX)
        - 4.3.2 Method 2: Android15 and Later (Cert)
    - 4.4 Code Modifications
        - 4.4.1 device/rockchip BoardConfig.mk Modifications
        - 4.4.2 u-boot defconfig Modifications
    - 4.5 AVB Key Flashing
        - 4.5.1 Method 1: Flashing via AVB Key Tool (Recommended for Production)
            - 4.5.1.1 Android14 and earlier (ATX)
            - 4.5.1.2 Android15 and Later (Cert)
        - 4.5.2 Method 2: Embed AVB Key in uboot Code and Flash with Firmware
            - 4.5.2.1 Apply the Patch in u-boot
            - 4.5.2.2 Extract Public Key
            - 4.5.2.3 Replace the Public Key
    - 4.6 Firmware Build
    - 4.7 Firmware Flashing
    - 4.8 Boot Verification
    - 4.9 AVB Strict Verification Unlock Procedure

# 1. Supported Chips

| Chip Platform | SecurityBoot | AVB | Minimum Android Version |
|---|---|---|---|
| RK3588 | Supported | Supported | Android12 RKR4 |
| RK3576 | Supported | Supported | Android14 |
| RK3572 | Supported | Supported | Android16 |
| RK3568 | Supported | Supported | Android11 RKR7 |
| RK3566 | Supported | Supported | Android11 RKR7 |
| RK3562 | Supported | Supported | Android13 |
| RK3326 | Supported | Supported | Android11 RKR7 |
| PX30 | Supported | Supported | Android11 RKR7 |

**Notes:**

- RK3399 series supports AVB only. For SecurityBoot, refer to the document "Rockchip_RK3399_User_Guide_SecurityBoot_And_AVB_EN.md"
- This document uses RK3576/RK3588/RK356x as examples. The steps for other chip platforms are similar, just replace the platform name accordingly.

# 2. Software Environment

- Android11 RKR7 and later versions support SecurityBoot and AVB
- See the table above for minimum Android version requirements for each chip platform

# 3. SecurityBoot Operation Steps

## 3.1 Build Environment Check

Verify that the fdtput version on the build server is 1.4.5

```
$ fdtput --version
Version: DTC 1.4.5
```

If the fdtput version is lower than 1.4.5, upgrade it with:

```
sudo apt-get install device-tree-compiler
```

## 3.2 Enter the u-boot Directory

```
cd u-boot
```

All operations below are performed in the u-boot directory.

## 3.3 Code Modifications

Enter the u-boot directory and open the defconfig file for the corresponding platform. Select the following configurations:

| Chip Platform | defconfig File |
| --- | --- |
| RK3588 | configs/rk3588_defconfig |
| RK3576 | configs/rk3576_defconfig |
| RK3572 | configs/rk3572_defconfig |
| RK3568/RK3566 | configs/rk3568_defconfig |
| RK3562 | configs/rk3562_defconfig |
| RK3326 | configs/rk3326_defconfig |
| PX30 | configs/px30_defconfig |

```
// Edit the defconfig file for the corresponding platform
vim configs/rk3576_defconfig
// Mandatory.
CONFIG_FIT_SIGNATURE=y
CONFIG_SPL_FIT_SIGNATURE=y
CONFIG_AVB_VBMETA_PUBLIC_KEY_VALIDATE=y

// Optional: Enable if RSA4096 keys are needed
CONFIG_FIT_ENABLE_RSA4096_SUPPORT=y

// Optional.
CONFIG_FIT_ROLLBACK_PROTECT=y       // boot.img anti-rollback
CONFIG_SPL_FIT_ROLLBACK_PROTECT=y   // uboot.img anti-rollback
```

**Note:** AVB keys default to RSA2048. To enable RSA4096, a patch is required. See: [https://redmine.rock-chips.com/issues/632050](https://redmine.rock-chips.com/issues/632050)

## 3.4 Generate Keys

Both rsa2048 and rsa4096 key generation are supported. Choose one of the following methods:
**Note:** The keys generated below are test keys for development only. **Production products must use independently generated release keys.** Do not use the default testkey in production.

Generate rsa2048 keys:

```
mkdir -p keys
../rkbin/tools/rk_sign_tool kk --bits 2048 --out .
cp privateKey.pem keys/dev.key && cp publicKey.pem keys/dev.pubkey
openssl req -batch -new -x509 -key keys/dev.key -out keys/dev.crt
```

Generate rsa4096 keys:

```
mkdir -p keys
../rkbin/tools/rk_sign_tool kk --bits 4096 --out .
cp privateKey.pem keys/dev.key && cp publicKey.pem keys/dev.pubkey
openssl req -batch -new -x509 -key keys/dev.key -out keys/dev.crt
```

**Important:** This step only needs to be performed once. Keep these keys securely. `--burn-key-hash` causes chip fusing, which is **irreversible**. Please confirm before proceeding.

## 3.5 Build and Sign

The following uses RK3576 as an example. Replace rk3576 with the corresponding platform name for other platforms:

```
./make.sh rk3576 --spl-new --rollback-index-uboot 1 --burn-key-hash
```

**Explanation:**

- - --spl-new: Mandatory. Repackage and sign the signed SPL
- - --rollback-index-uboot : Optional. Sets the rollback version number. Required only if rollback protection is enabled in the "Code Modifications" section.
- - --burn-key-hash: Optional. When enabled, the chip will be fused upon first boot after flashing firmware.

**If the following error occurs:**

```
Can't load XXXXXX//.rnd into RNG
```

Run:

```
touch ~/.rnd
```

### 3.5.1 FIT Independent Signing

Some customers prefer not to use the above build-time signing method and need a server-based independent signing approach. For Android platforms, only loader and uboot.img need to be signed.

### 3.5.2 Signing Flow

**For firmware build engineers:**

1. Refer to the SecurityBoot section to enable the relevant configuration options.

2. Build:

```
./make.sh rk3576 --spl-new --no-sign       # Must append --spl-new --no-sign
```

The following files will be generated:

```
uboot.img
rk3576_spl_loader_v1.10.108.bin
fit/fit_signcfg/
```

3. Send the three generated files to the signing engineer or signing server.

**Note:** The generated loader, uboot, and fit_signcfg have a strict matching relationship. Do not mix them with firmware of the same name compiled at different times, or the signing script will report errors.

**For signing engineers or signing server:**

1. Create a working directory (name can be arbitrary):

```
$ mkdir workspace
$ cd workspace
$ mkdir keys src output
```

2. Download the rkbin repository and copy or symlink the tools directory to the workspace directory:

```
$ cp -r ~/rkbin/tools/ ./
```

3. Place the RSA key pair in the keys directory, and the firmware to be signed along with the fit_signcfg directory in the src directory.

The loader, uboot and fit_signcfg are mandatory. Example:

```
$ tree
.
├── keys
│   ├── dev.crt
│   ├── dev.key
│   └── dev.pubkey
├── output
├── src
│   ├── fit_signcfg
│   │   ├── MINIALL.ini
│   │   ├── sign.readonly_config
│   │   └── test_images.tar.gz
│   ├── rk3576_spl_loader_v1.10.108.bin
│   └── uboot.img
└── tools
    ├── boot_merger
    ├── burner_image_kits
    │   ├── make.sh
    │   └── make_slc_nand.sh
    ......
```

4. Execute the signing:

```
$ ./tools/fit-sign.sh --key-dir keys/ --src-dir src/ --out-dir output/
```

The signing process does not modify any source files in the src directory. After successful signing, all signed firmware is generated in the output directory:

```
$ tree output/
output/
├── rk3576_idblock_v1.10.108.img
├── rk3576_spl_loader_v1.10.108.bin      // Equivalent to MiniLoaderAll.bin
└── uboot.img
```

### 3.5.3 Signing Script

The independent signing tools have been committed to the rkbin repository. The script called by developers is: ./tools/fit-sign.sh.

```
$ ./tools/fit-sign.sh -h

Usage:
    ./tools/fit-sign.sh [args]

Args:
    --key-dir                   <dir>                          | Mandatory
    --src-dir                   <dir>                          | Mandatory
    --out-dir                   <dir>                          | Mandatory
    --burn-key-hash                                            | Optional
    --rollback-index            <image1 n1> <image2 n2> ...    | Optional
    --version                   <image1 n1> <image2 n2> ...    | Optional
```

- - --key-dir: Mandatory. Points to the RSA key directory.
- --src-dir: Mandatory. Points to the firmware directory to be signed. Must include loader, uboot, and fit_signcfg.
- --out-dir: Mandatory. Points to the output directory for signed firmware.
- --burn-key-hash: Optional. Enables SPL to burn the RSA public key hash to OTP.
- --rollback-index: Optional. Specifies the anti-rollback version number for the corresponding firmware, in decimal.
- --version: Optional. Specifies the regular file version number for the corresponding firmware, in decimal.

Example:

```
./tools/fit-sign.sh --key-dir keys/ --src-dir src/ --out-dir output/ --version
uboot.img 1 boot.img 3 --rollback-index uboot.img 3 boot.img 5
```

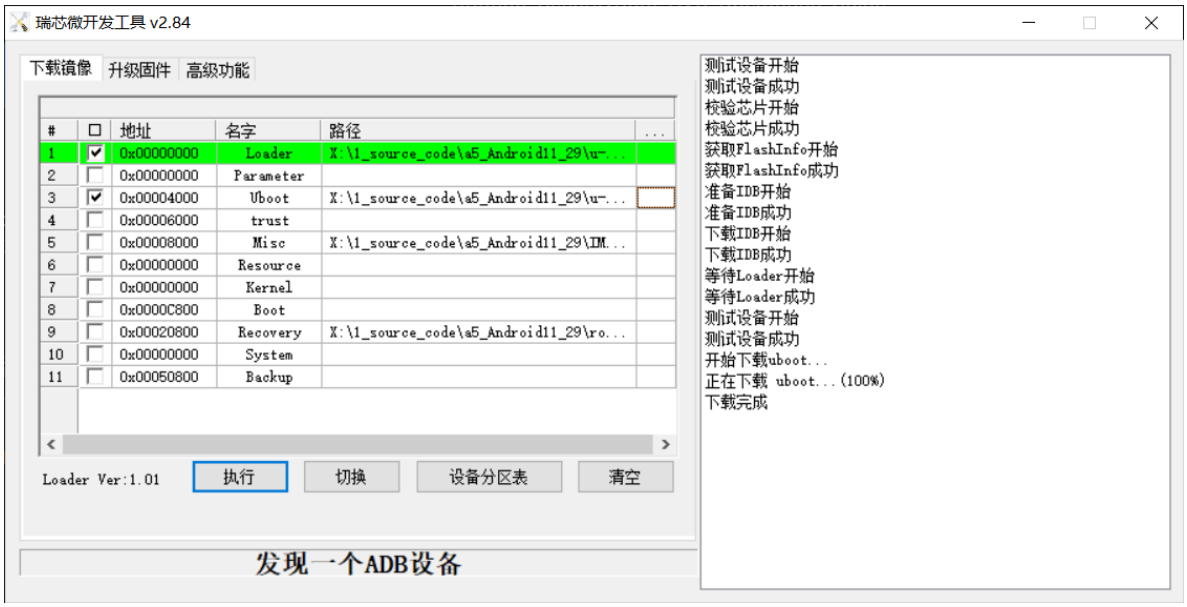
## 3.6 Build Complete Firmware

Build the remaining firmware using the normal compilation method (uboot and loader have already been built and do not need to be recompiled). For example:

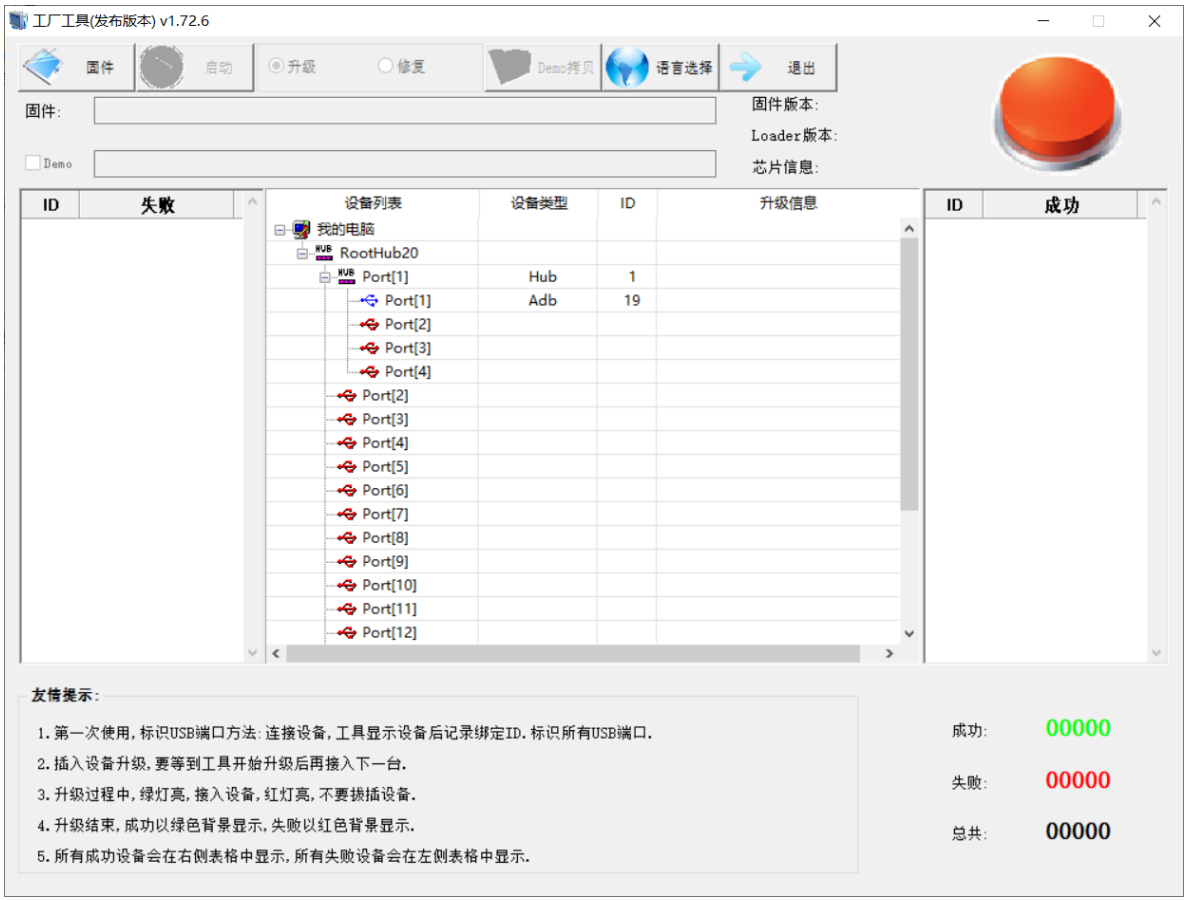
```
source build/envsetup.sh
lunch rk3576_u-userdebug
./build.sh -ACKup
```

## 3.7 Firmware Flashing

During development and verification, use AndroidTool to flash uboot and loader separately:




For mass production, use the factory tool to flash the update.img from the previous build step:




## 3.8 Verify Fusing Success

### 3.8.1 Check via UART Boot Log

```
### Verified-boot: 0   // Firmware signed but chip is not fused (1=fused), no
hash validation
sha256,rsa2048:dev+
rollback index: 1 >= 0(min), OK   // Rollback version
// Below are uboot integrity checks
## Checking atf-1 0x00040000 ... sha256+ OK
## Checking uboot 0x00a00000 ... sha256+ OK
## Checking fdt 0x00b2a018 ... sha256+ OK
## Checking atf-2 0xfdcc9000 ... sha256+ OK
## Checking atf-3 0xfdcd0000 ... sha256+ OK
## Checking optee 0x00200000 ... sha256+ OK
```

### 3.8.2 Check by Flashing Unsigned Loader and Uboot

Flashing an unsigned loader or uboot will cause the system verification to fail, preventing the device from booting.

# 4. Android Verified Boot (AVB) Operation Steps

## 4.1 About AVB Key Management

Rockchip platforms use different AVB key management mechanisms depending on the Android version:

| Android Version | Key Management | Key File Prefix | Description |
| --- | --- | --- | --- |
| Android15 and later | Cert | testkey_cert* / *cert** | Certificate-based authentication |
| Android14 and earlier | ATX | testkey_atx* / *atx** | Legacy ATX authentication |

**Note:** The key file names and configuration option names differ between ATX and Cert mechanisms. Select the corresponding section based on the actual Android version being used.

## 4.2 Build avbtool

```
mmma external/avb/ -j16
```

After compilation, the following is generated:

```
out/host/linux-x86/bin/avbtool
```

## 4.3 Generate permanent_attributes.bin

Choose the corresponding generation method based on the Android version:

### 4.3.1 Method 1: Android14 and Earlier (ATX)

- **Modify Product ID**

```
cd external/avb/test
```

```
-echo 00000000000000000000000000000000 | xxd -r -p - atx_product_id.bin
+echo 00000000000000000000000000000123 | xxd -r -p - atx_product_id.bin
```

```
cd -
```

**Note:** The product ID is 16 bytes. You can define your own value.

- **Generate atx_permanent_attributes.bin**

```
cd external/avb/test/data
../avb_atx_generate_test_data
cd -
```

After executing the above steps, the following files are generated in external/avb/test/data:

- atx_permanent_attributes.bin
- atx_metadata.bin
- testkey_atx_pik.pem
- testkey_atx_prk.pem
- testkey_atx_psk.pem

**Notes:**

- A default pem file exists in the system. To regenerate, delete the default files first and then run the steps above. It is recommended that customers regenerate their own pem files.
- This step only needs to be performed once per product. Keep the generated files securely for use in the following steps.

### 4.3.2 Method 2: Android15 and Later (Cert)

- **Modify Product ID**

```
cd external/avb/test
```

```
-echo 00000000000000000000000000000000 | xxd -r -p - cert_product_id.bin
+echo 00000000000000000000000000000123 | xxd -r -p - cert_product_id.bin
```

```
cd -
```

**Note:** The product ID is 16 bytes. You can define your own value.

- **Generate cert_permanent_attributes.bin**

```
cd external/avb/test/data
../avb_cert_generate_test_data
cd -
```

After executing the above steps, the following files are generated in external/avb/test/data:

- cert_permanent_attributes.bin
- cert_metadata.bin
- testkey_cert_pik.pem
- testkey_cert_prk.pem
- testkey_cert_psk.pem
- testkey_cert_puk.pem

**Notes:**

- A default pem file exists in the system. To regenerate, delete the default files first and then run the steps above. It is recommended that customers regenerate their own pem files.
- This step only needs to be performed once per product. Keep the generated files securely for use in the following steps.

# 4.4 Code Modifications

## 4.4.1 device/rockchip BoardConfig.mk Modifications

Navigate to the device directory for the corresponding chip platform. Using RK3576 as an example:

```
cd device/rockchip/rk3576
```

```
 BOARD_AVB_ENABLE := true    // Enable AVB
 BOARD_AVB_ALGORITHM := SHA256_RSA4096  // Configure signature algorithm
+# For Android14 and earlier (ATX) use these:
+#BOARD_AVB_KEY_PATH := external/avb/test/data/testkey_atx_psk.pem
+#BOARD_AVB_METADATA_BIN_PATH := external/avb/test/data/atx_metadata.bin
+# For Android15 and later (Cert) use these:
+BOARD_AVB_KEY_PATH := external/avb/test/data/testkey_cert_psk.pem
+BOARD_AVB_METADATA_BIN_PATH := external/avb/test/data/cert_metadata.bin
 #BOARD_AVB_ROLLBACK_INDEX := 5   // Configure anti-rollback, disabled by
default
```

**Reference for device directories by chip platform:**

| Chip Platform | device directory |
| --- | --- |
| RK3588 | device/rockchip/rk3588 |
| RK3576 | device/rockchip/rk3576 |
| RK3572 | device/rockchip/rk3572 |
| RK3568/RK3566 | device/rockchip/rk356x |
| RK3562 | device/rockchip/rk3562 |
| RK3326 | device/rockchip/rk3326 |
| PX30 | device/rockchip/px30 |

### 4.4.2 u-boot defconfig Modifications

```
cd u-boot
```

```
+# For Android14 and earlier (ATX) use:
+CONFIG_AVB_LIBAVB_ATX=y
+# For Android15 and later (Cert) use:
+CONFIG_AVB_LIBAVB_CERT=y
 CONFIG_AVB_LIBAVB_USER=y
 CONFIG_RK_AVB_LIBAVB_USER=y
+CONFIG_RK_AVB_LIBAVB_ENABLE_ATH_UNLOCK=y
+CONFIG_AVB_VBMETA_PUBLIC_KEY_VALIDATE=y
+CONFIG_ANDROID_AVB_ROLLBACK_INDEX=y // Anti-rollback, enable only if needed
```

**Note:** Select either CONFIG_AVB_LIBAVB_ATX or CONFIG_AVB_LIBAVB_CERT based on the Android version. Both cannot be enabled simultaneously.

```
cd -
```

## 4.5 AVB Key Flashing

### 4.5.1 Method 1: Flashing via AVB Key Tool (Recommended for Production)

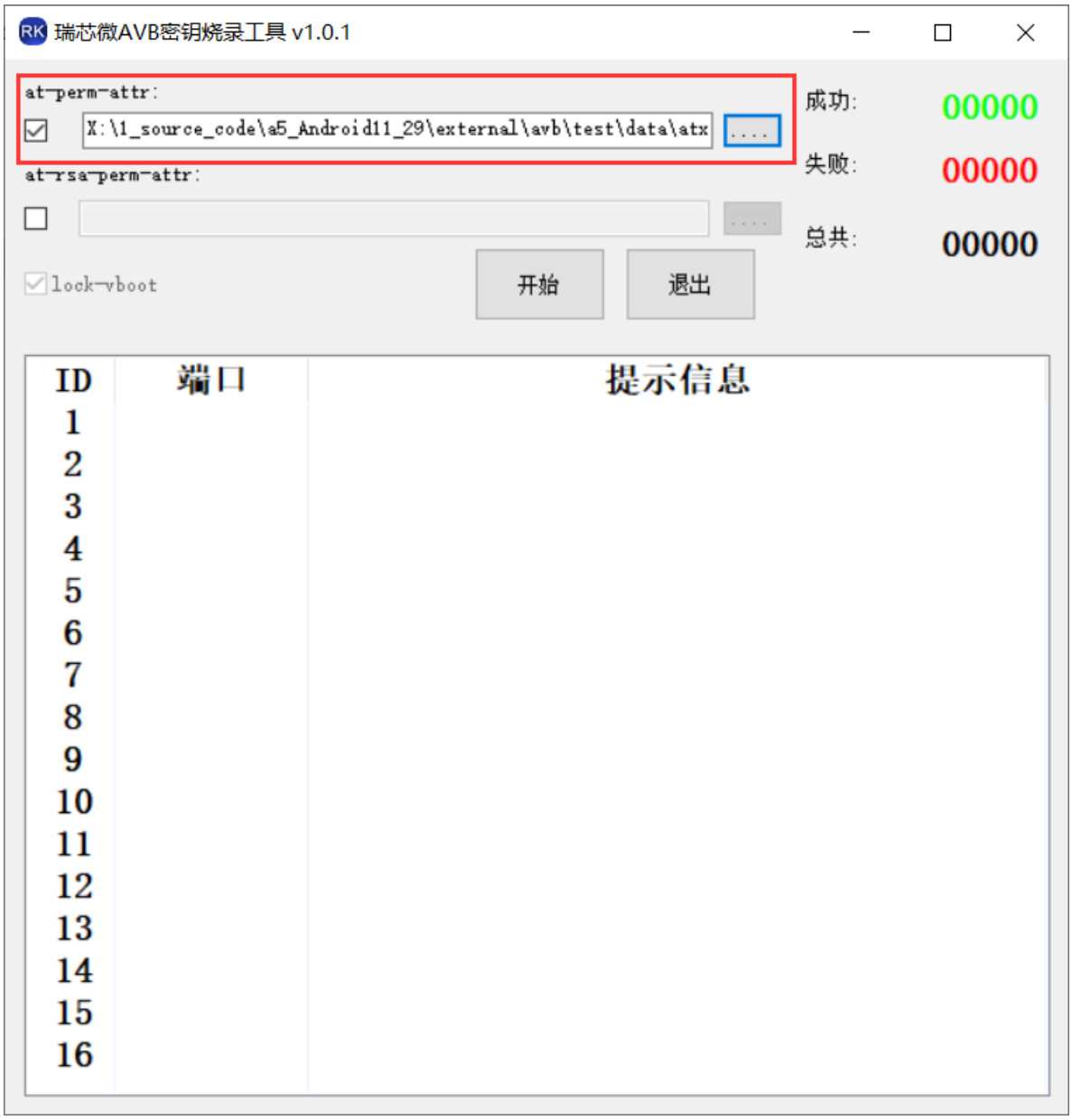
Select the corresponding flashing tool based on the Android version:

#### 4.5.1.1 Android14 and earlier (ATX)

Flashing tool: AvbKeyWriter (RKTools/windows/AvbKeyWriter-v1.0.1.7z)
Source file: atx_permanent_attributes.bin
Procedure:

- Check "at-perm-attr"
- Importatx_permanent_attributes.bin
- Put the device into loader mode

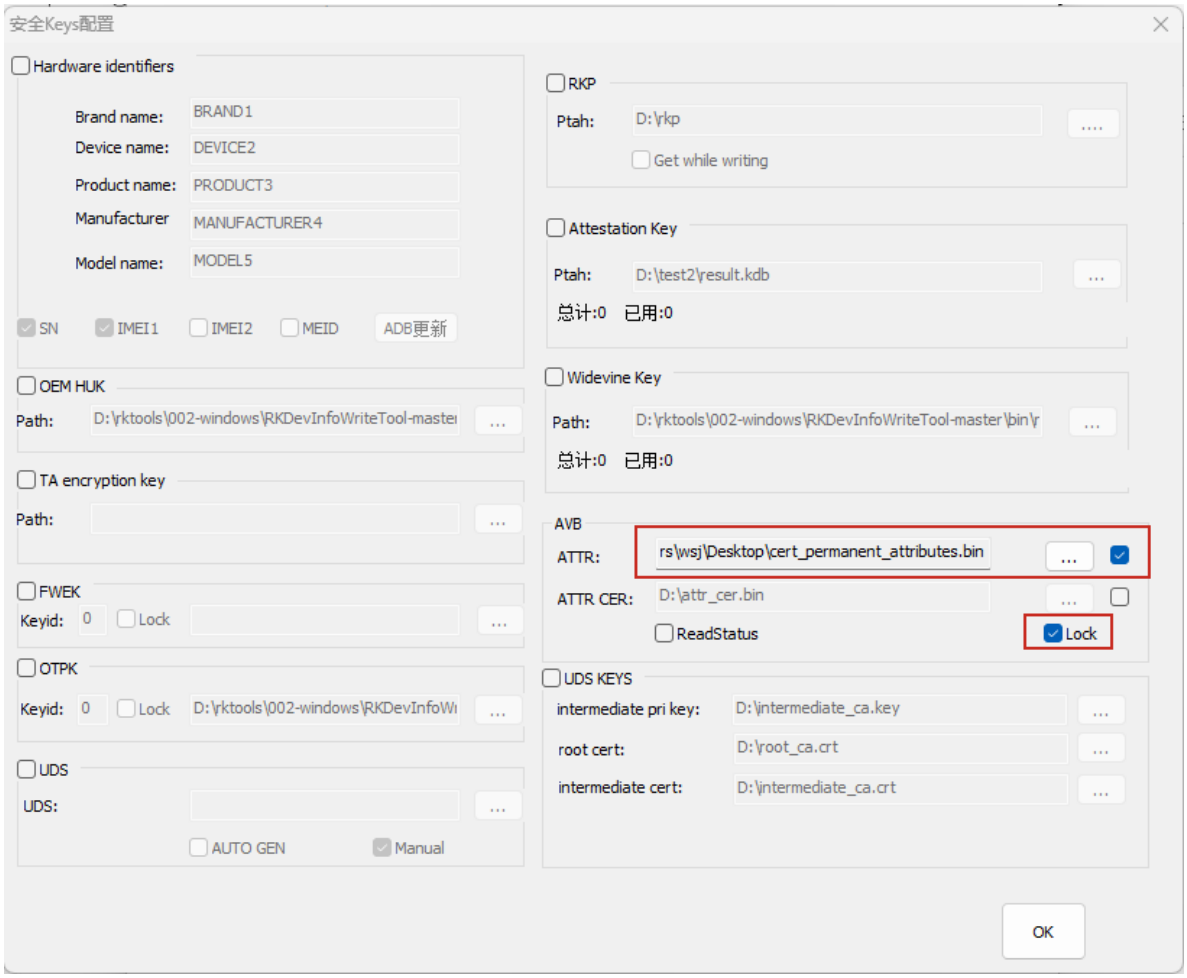
- Click "Flash" button

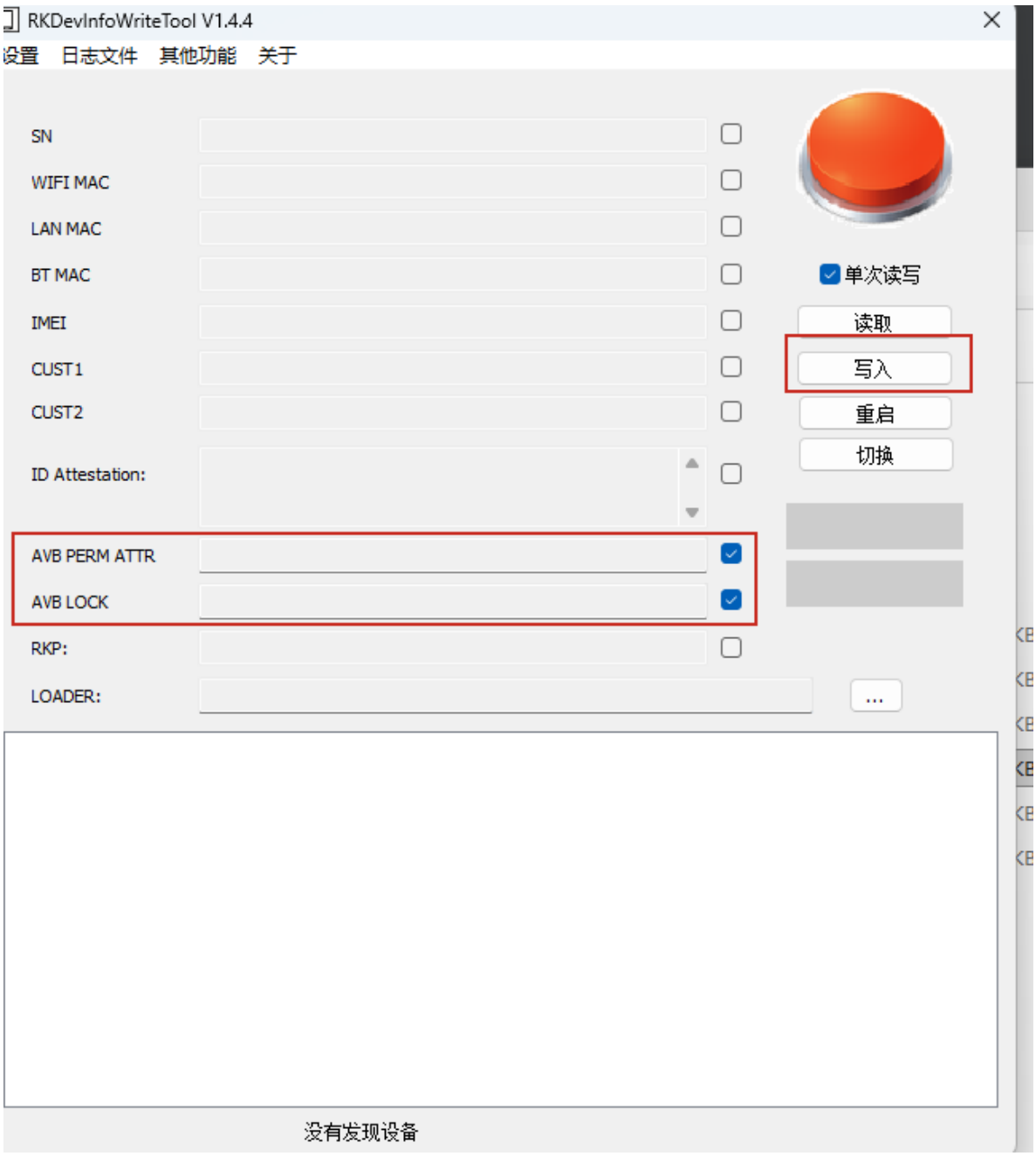



#### 4.5.1.2 Android15 and Later (Cert)

Flashing tool: RKDevInfoWriteTool (RKTools/windows/RKDevInfoWriteTool-v1.4.4_20260522 and later)
Source file: cert_permanent_attributes.bin
Procedure:

- Tool => Settings => Security Key Configuration, check "AVB ATTR"
- Importcert_permanent_attributes.bin
- Put the device into loader mode
- Click "Flash" button


安全Keys配置
Hardware identifiers
Brand name:
BRAND1
Device name:
DEVICE2
Product name:
PRODUCT3
Manufacturer
MANUFACTURER4
Model name:
MODEL5
SN
IMEI1
IMEI2
MEID
ADB更新
OEM HUK
Path:
D:\rktools\002-windows\RKDevInfoWriteTool-maste
TA encryption key
Path:
FWEK
Keyid:
0
Lock
OTPK
Keyid:
0
Lock
D:\rktools\002-windows\RKDevInfoWi
UDS
UDS:
AUTO GEN
Manual
RKP
Ptah:
D:\rkp
Get while writing
Attestation Key
Ptah:
D:\test2\result.kdb
总计:0 已用:0
Widevine Key
Path:
D:\rktools\002-windows\RKDevInfoWriteTool-master\bin\r
总计:0 已用:0
AVB
ATTR:
rs\wsj\Desktop\cert_permanent_attributes.bin
ATTR CER:
D:\attr_cer.bin
ReadStatus
Lock
UDS KEYS
intermediate pri key:
D:\intermediate_ca.key
root cert:
D:\root_ca.crt
intermediate cert:
D:\intermediate_ca.crt
OK




### 4.5.2 Method 2: Embed AVB Key in uboot Code and Flash with Firmware

In this approach, the key is embedded in the uboot.img firmware rather than being written to the secure partition. If boot.img is replaced, the key becomes invalid. Therefore, SecurityBoot must be enabled to prevent tampering with the uboot partition.

**Note:** This method does not support strict AVB verification unlock. To use the unlock feature, use Method 1.

#### 4.5.2.1 Apply the Patch in u-boot

Select the patch based on the uboot branch:

| Chip Platform | uboot Branch | Patch File |
| --- | --- | --- |
| RK3572 | V5 | 0001-avb-add-embedded-key-for-v5.patch |
| Other chips | next-dev | 0001-avb-add-embedded-key.patch |

Patch path: RKDocs/common/security/patch/u-boot/

```
cd u-boot
# For RK3572 use:
# git am RKDocs/common/security/patch/u-boot/0001-avb-add-embedded-key-for-
v5.patch
# For other chips use:
git am RKDocs/common/security/patch/u-boot/0001-avb-add-embedded-key.patch
cd -
```

#### 4.5.2.2 Extract Public Key

Select the corresponding key file based on the Android version:

Android14 and earlier (ATX):

```
avbtool extract_public_key --key external/avb/test/data/testkey_atx_psk.pem --
output avb_root_pub.bin
xxd -i avb_root_pub.bin > external/avb/test/data/avb_root_pub.h
```

Android15 and later (Cert):

```
avbtool extract_public_key --key external/avb/test/data/testkey_cert_psk.pem --
output avb_root_pub.bin
xxd -i avb_root_pub.bin > external/avb/test/data/avb_root_pub.h
```

#### 4.5.2.3 Replace the Public Key

Replace the avb_root_pub array in u-boot/lib/avb/libavb_user/avb_ops_user.c with the extracted public key from external/avb/test/data/avb_root_pub.h:

```
cd u-boot
vim lib/avb/libavb_user/avb_ops_user.c
```

```
/*
* Internal builds use testkey_rsa4096.pem
* OEM should replace this Array with public key used to sign vbmeta.img
*/
static const char avb_root_pub[] = {
0x00, 0x00, 0x10, 0x00, 0x55, 0xd9, 0x04, 0xad, 0xd8, 0x04, 0xaf, 0xe3,
0xd3, 0x84, 0x6c, 0x7e, 0x0d, 0x89, 0x3d, 0xc2, 0x8c, 0xd3, 0x12, 0x55,
...
};
```

```
cd -
```

## 4.6 Firmware Build

After completing the above steps, build the complete firmware. Using RK3576 as an example:

- **With SecurityBoot**

  uboot must be compiled separately first for SecurityBoot. See the SecurityBoot section above.

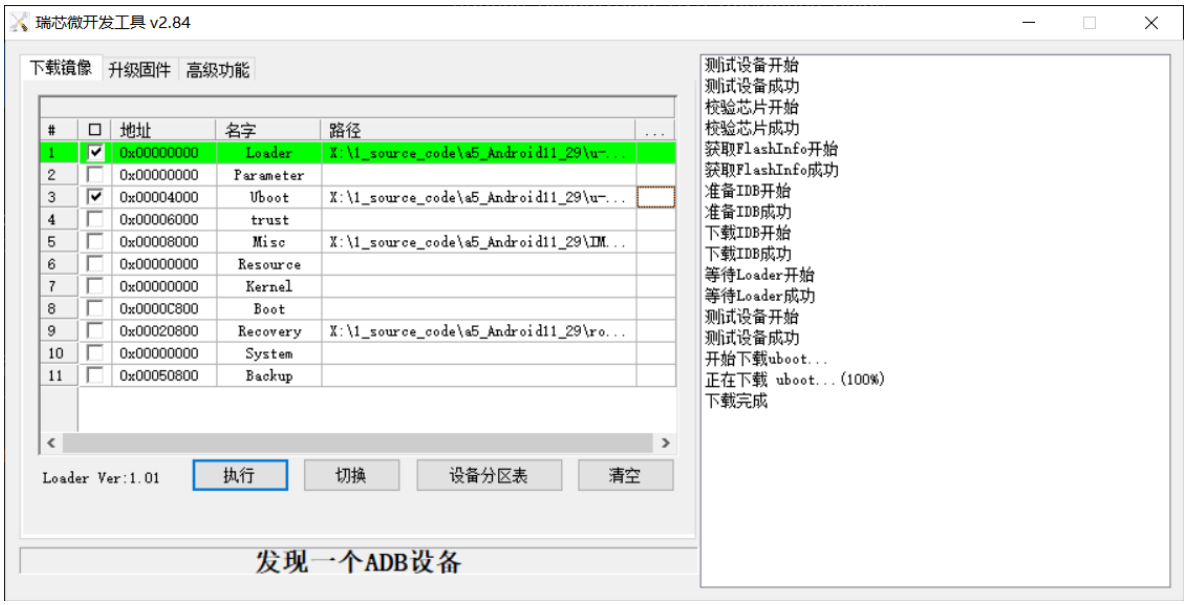
```
source build/envsetup.sh
lunch rk3576_u-userdebug
./build.sh -ACKup
```

- **Without SecurityBoot**

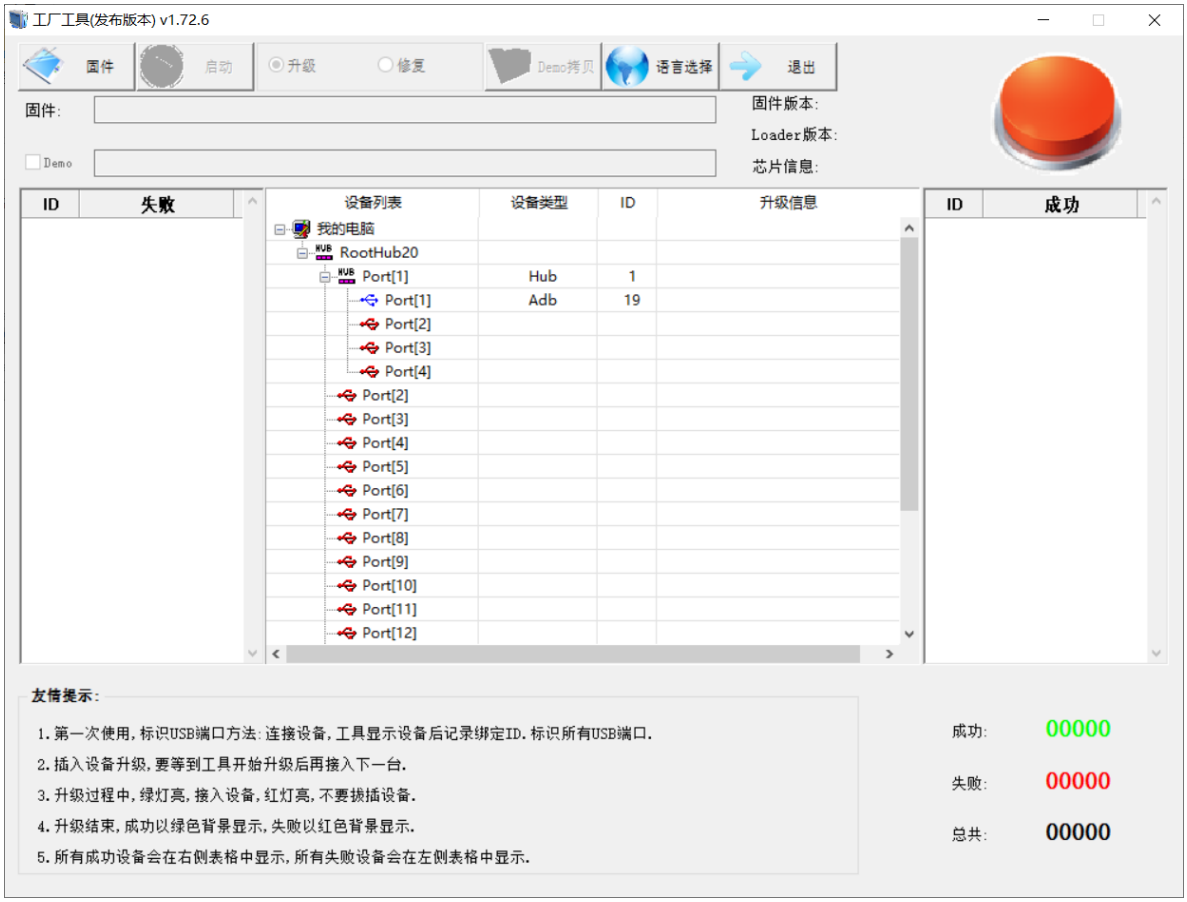
```
source build/envsetup.sh
lunch rk3576_u-userdebug
./build.sh -ACKUup
```

## 4.7 Firmware Flashing

During development, use AndroidTool to flash the complete firmware




For mass production, use the factory tool to flash the update.img from the previous build step:




## 4.8 Boot Verification

- **Check uboot Boot Log**

After following the above steps, the following prints should appear during the u-boot phase in the UART serial log:

**Android14 and earlier (ATX, without SecurityBoot):**

```
Vboot=0, AVB images, AVB verify
read_is_device_unlocked() ops returned that device is LOCKED
ANDROID: Hash OK
```

**Android15 and later (Cert, with SecurityBoot):**

```
Vboot=1, SecureBoot enabled, AVB verify
read_is_device_unlocked() ops returned that device is LOCKED
preloaded(s): pre-full image from 'recovery' at 0xf51f85c0 - 0xf9ea65c0
ANDROID: sha1(a1ea6f6f4f...) + OK
```

- Flashing a non-AVB firmware or other firmware will cause the device to fail to boot.

## 4.9 AVB Strict Verification Unlock Procedure

- Enter fastboot mode on the device, then on the host PC:

```
adb reboot bootloader
fastboot oem at-get-vboot-unlock-challenge
fastboot get_staged raw_unlock_challenge.bin
```

```
-pc:~/GMS-Suite/tools$ adb reboot bootloader
-pc:~/GMS-Suite/tools$
-pc:~/GMS-Suite/tools$
-pc:~/GMS-Suite/tools$ fastboot devices
J7U      Android Fastboot

-pc:~/GMS-Suite/tools$
-pc:~/GMS-Suite/tools$
-pc:~/GMS-Suite/tools$ fastboot oem at-get-vboot-unlock-challenge
 0.015s]
d. Total time: 0.016s
-pc:~/GMS-Suite/tools$
-pc:~/GMS-Suite/tools$
-pc:~/GMS-Suite/tools$ fastboot get_staged raw_unlock_challenge.bin
ng 'raw_unlock_challenge.bin'                OKAY [  0.002s]
d. Total time: 0.002s
-pc:~/GMS-Suite/tools$
-pc:~/GMS-Suite/tools$
-pc:~/GMS-Suite/tools$
-pc:~/GMS-Suite/tools$
-pc:~/GMS-Suite/tools$
-pc:~/GMS-Suite/tools$ ls -l
460
r-- 1 hcq hcq 2876596 4月  14 19:57 OverlayTools.apk
r-- 1 hcq hcq      52 4月  27 14:21 raw_unlock_challenge.bin
r-- 1 hcq hcq   19692 2月  17 09:27 rockchip_selinux_tools
r-x 1 hcq hcq 3703111 2月  16 20:41 SSR.apk
r-x 1 hcq hcq     580 2月  16 20:41 ssr.txt
-pc:~/GMS-Suite/tools$
```

This retrieves data containing the version, Product ID, and a 16-byte random number. Extract the random number to generate raw_unlock_challenge.bin.

- Generate unlock_credential.bin using avbtool

  Select the corresponding product_id file based on the Android version:

Android14 and earlier (ATX):

```
python avb-challenge-verify.py raw_unlock_challenge.bin atx_product_id.bin
```

Android15 and later (Cert):

```
python avb-challenge-verify.py raw_unlock_challenge.bin cert_product_id.bin
```

```
python avbtool make_unlock_credential --output=unlock_credential.bin --
intermediate_key_certificate=external/avb/test/data/testkey_cert_pik.pem --
unlock_key_certificate=external/avb/test/data/testkey_cert_puk.pem --
challenge=unlock_challenge.bin --
unlock_key=external/avb/test/data/testkey_cert_psk.pem
```

The implementation of avb-challenge-verify.py is as follows. Save this code as avb-challenge-verify.py:

```
#!/usr/bin/env python
\"\"\"This is a test module for getting unlock_challenge.bin\"\"\"
import sys
```

```
from hashlib import sha256

def challenge_verify():
    if len(sys.argv) != 3:
        print("Usage: avb-challenge-verify.py [challenge_file]
[product_id_file]")
        return
    if sys.argv[1] in ("-h", "--h"):
        print("Usage: avb-challenge-verify.py [challenge_file]
[product_id_file]")
        return
    try:
        with open(sys.argv[1], "rb") as f:
            challenge_data = f.read(52)
        with open(sys.argv[2], "rb") as f:
            product_id_data = f.read(16)
        product_id_hash = sha256(product_id_data).digest()
        print("The challenge version is %d" % challenge_data[0])
        if product_id_hash != challenge_data[4:36]:
            print("Product id verify error!")
            return
        with open("unlock_challenge.bin", "wb") as f:
            f.write(challenge_data[36:52])
        print("Success!")
    except Exception as e:
        print("Error: %s" % str(e))

if __name__ == "__main__":
    challenge_verify()
```

- Flash unlock_credential.bin, on the host PC:

```
fastboot stage unlock_credential.bin
fastboot oem at-unlock-vboot
```

**Important:** The device must remain in fastboot mode throughout this process. Do not power off, shut down, or reboot after step 1, as the device stores the generated random number in memory. Power loss will cause the random number to be lost, and subsequent challenge signature verification will fail.

If enabled:

```
CONFIG_MISC=y
CONFIG_ROCKCHIP_EFUSE=y
CONFIG_ROCKCHIP_OTP=y
```

the CPUID will be used as the challenge number. The CPUID is unique to each device and persists across reboots, so the generated unlock_credential.bin can be reused. This eliminates the need to regenerate unlock_challenge.bin and unlock_credential.bin each time. The unlock steps become:

```
fastboot oem at-get-vboot-unlock-challenge
fastboot stage unlock_credential.bin
fastboot oem at-unlock-vboot
```